

Computer Vision - Spring 2026

Smart AI Surveillance and Shape-Based Object Analysis System

A combined real-time computer vision system for object detection, motion analysis, tracking, counting, and shape-based understanding.

PROJECT TYPE

Combined Intelligent Surveillance System

TECHNOLOGIES

YOLOv8 - OpenCV - Motion Detection - Hough Transform

UNDER SUPERVISION OF

Dr. Yasser Fouaad
Eng. Rashed Mahmoud

TEAM MEMBERS

Nour Emad Abdallah - 2305343
Mostafa Mohamed Gamal - 2305084
Mohamed Essam Ghaith - 2305378

Abstract

This project presents an intelligent traffic surveillance and shape-based object analysis system. The system combines deep learning object detection with classical computer vision techniques to analyze videos in real time. It detects objects, analyzes motion, extracts object edges, detects lines and circles, tracks objects across frames, and counts objects entering or leaving the scene.

The project combines the two required ideas into one integrated system. The first part focuses on smart shape-based object counting and object analysis, while the second part focuses on motion-aware surveillance. By combining both parts, the final system becomes more powerful because it can detect objects using YOLOv8 and also analyze motion and object structure using OpenCV techniques.

Project Objectives

Real-Time Detection
Detect multiple objects such as persons, bicycles, cars, buses, trucks, and motorcycles.

Motion Awareness
Detect moving areas using background subtraction and morphology operations.

Shape Analysis
Use Canny Edge Detection and Hough Transform to extract lines and circular shapes.

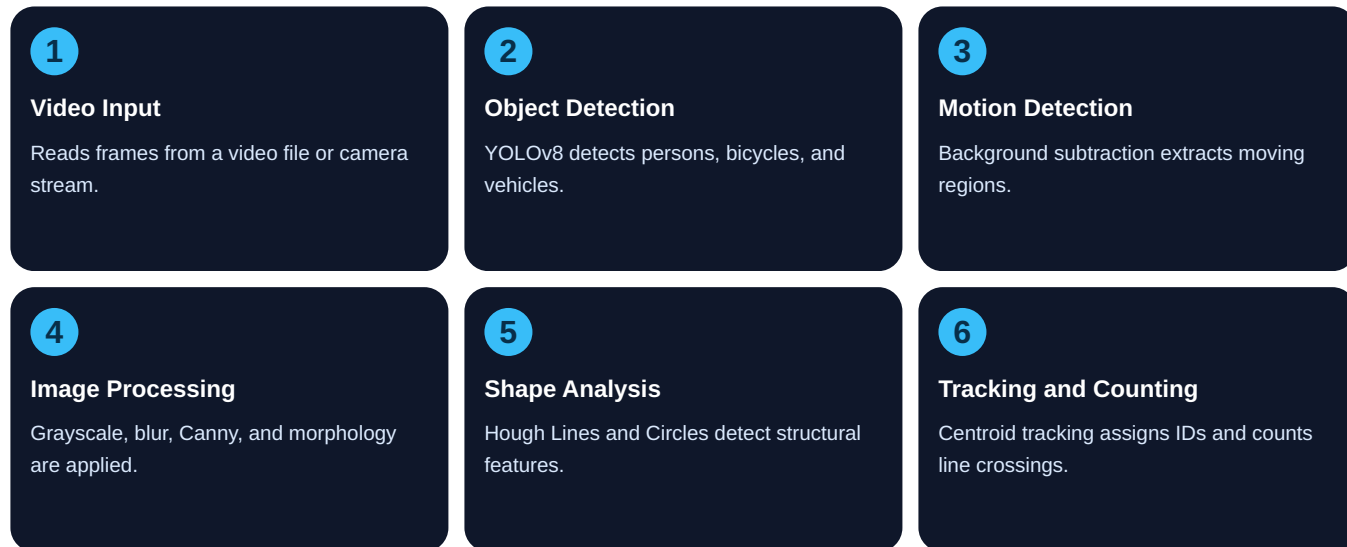
Counting and Tracking
Track objects using centroid tracking and count entering/leaving objects using a virtual line.

Tools and Technologies

Technology	Purpose in the Project
Python	Main programming language used to implement the full system.
OpenCV	Video processing, drawing, motion detection, morphology, Canny, Hough Transform, and GUI controls.
YOLOv8	Real-time object detection and classification.
NumPy	Matrix operations and image processing support.
Centroid Tracking	Assigns object IDs and follows detected objects across frames.

System Architecture

The system is designed using multiple modules. Each module performs a specific task, then passes its output to the next stage. This modular structure makes the system easier to understand, improve, and explain during discussion.



Implementation Details

YOLOv8 Object Detection

YOLOv8 is used because it is fast and suitable for real-time detection. The lightweight model is used to keep the system responsive while detecting multiple object classes. The system filters detections using confidence, IOU, and minimum box area to reduce noisy results.

Motion Detection

Motion is detected using background subtraction. The output mask is cleaned using thresholding, opening, closing, and dilation. This helps remove small noise and improves the quality of detected moving regions.

Tracking and Counting

The centroid tracker calculates the center point of each bounding box and compares positions across frames. If an object crosses the counting line, the system updates the entering or leaving counter. Counted object IDs are stored to reduce double counting.

Shape-Based Analysis

Canny Edge Detection extracts strong edges inside detected objects. Hough Transform can then identify straight lines and circular shapes. This supports the analysis of bicycles, vehicles, and object structures.

Requirement Coverage

Requirement	Implemented Solution
Detect multiple objects	YOLOv8 detects persons, bicycles, cars, buses, trucks, and motorcycles.
Apply grayscale and blur	Each object region can be processed using grayscale conversion and Gaussian Blur.
Canny Edge Detection	Canny is used to extract object boundaries and support shape understanding.
Hough Lines and Circles	Hough Transform is used for line and circle detection, especially useful for vehicle and bicycle structure analysis.
Motion Detection	Background subtraction identifies moving regions in the video.
Tracking	Centroid tracking assigns IDs and tracks objects between frames.
Counting	A virtual counting line counts entering and leaving objects while reducing double counting.
Display	The interface displays bounding boxes, labels, counters, and optional motion mask output.
Controls	Trackbars control confidence, IOU, image size, minimum area, line position, and display options.

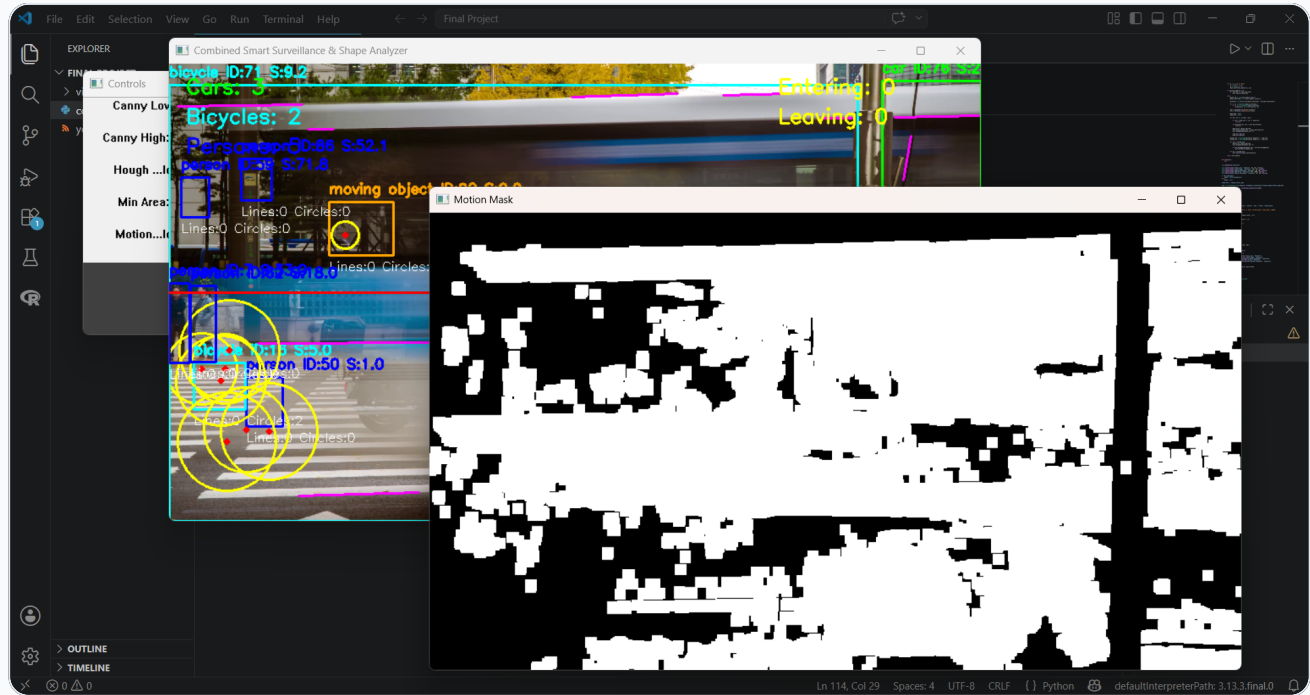
Controller Explanation

Controller	Function	Recommended Value
YOLO Conf	Minimum confidence required to accept a detection.	45-55
YOLO IOU	Controls duplicate bounding box suppression.	40-50
Image Size	Input image size for YOLO. Larger is more accurate but slower.	960
Min Box Area	Ignores very small detections to reduce clutter.	500-1000
Motion Threshold	Controls sensitivity of the binary motion mask.	200-240
Motion Min Area	Ignores small moving regions caused by noise.	700-2000
Counting Line	Moves the virtual line used for entering/leaving counts.	50
Show IDs / Centers / Motion	Optional display controls used during explanation and debugging.	Off for clean demo

Experimental Results

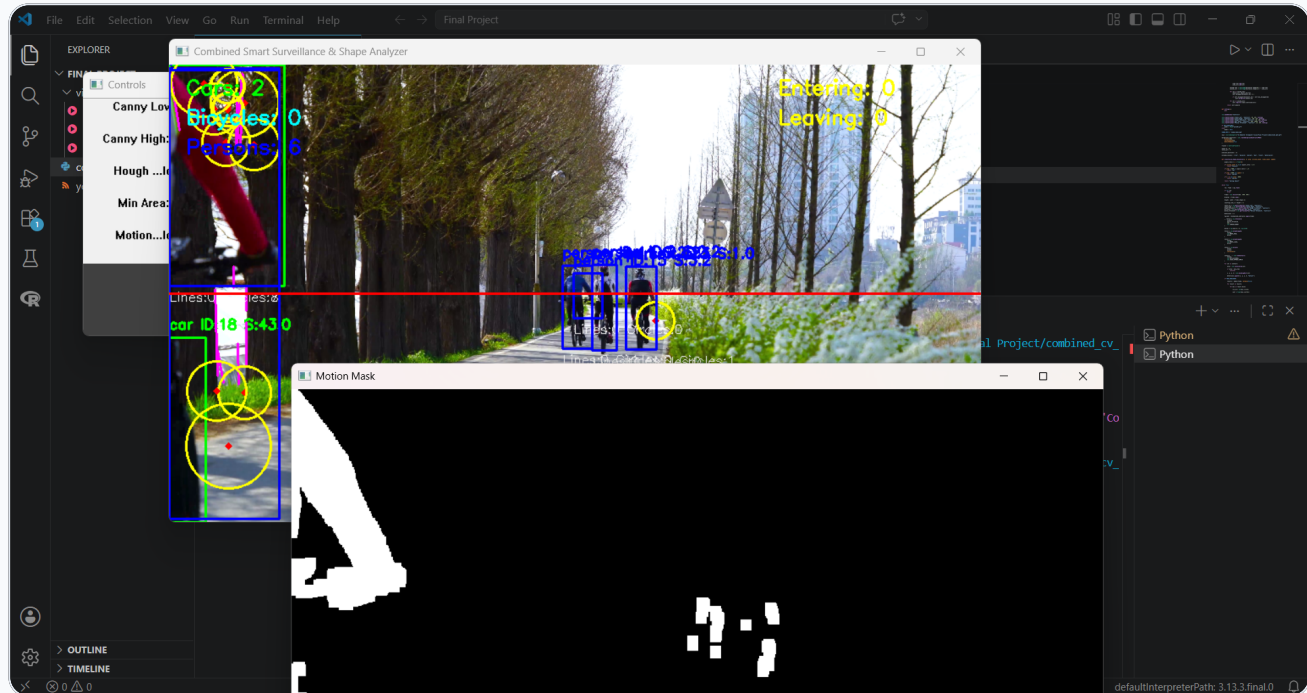
The following screenshots are taken from the actual project output. They show the detection results, object labels, counters, and surveillance output generated by the system.

Figure 1



Bicycle and person detection with visual bounding boxes and live class counters.

Figure 2

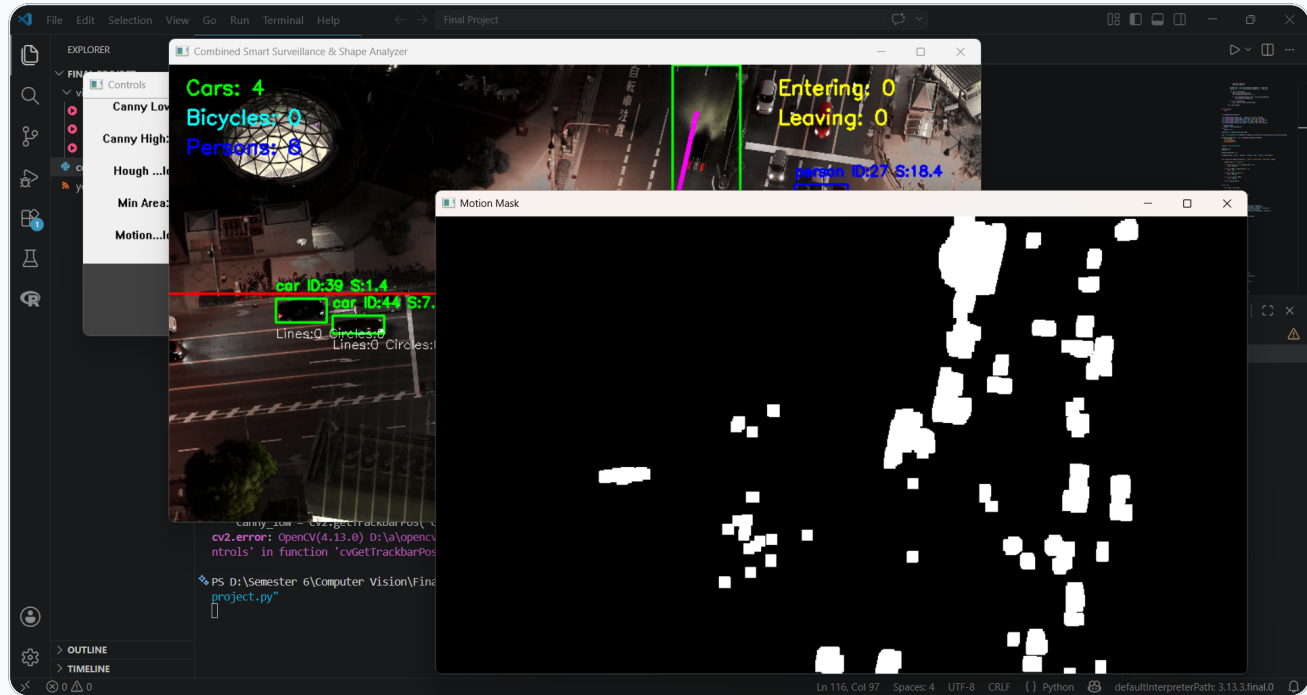


Traffic scene analysis showing object detection, tracking, and real-time surveillance output.

The screenshot shows a software application titled "Combined Smart Surveillance & Shape Analyzer". The main window displays a video of a person on a bicycle. The video is overlaid with several colored bounding boxes and labels. In the top left corner, there is a status bar showing "Cdns: 1", "Bicycles: 0", and "Persons: 5". In the top right corner, there is a status bar showing "Entering: 0" and "Leaving: 0". The video is divided into two sections by a horizontal red line. The top section shows a person on a bicycle with a blue bounding box and a red dot. The bottom section shows a person on a bicycle with a blue bounding box and a red dot. A smaller window titled "Motion Mask" is open in the foreground, showing a white silhouette of a person on a bicycle against a black background. The application interface includes a menu bar with "File", "Edit", "Selection", "View", "Go", "Run", "Terminal", and "Help". On the left side, there is an "EXPLORER" panel with a tree view showing "FIN", "Controls", "Canny Low", "Canny High", "Hough ...", "Min Area", and "Motion...". On the right side, there is a "TIMELINE" panel with a timeline view showing "OUTLINE" and "TIMELINE".

Bicycle wheel and structure analysis using shape-based detection concepts.

Figure 4



Vehicle monitoring example with object counters and motion analysis support.

Discussion

The results show that the system can detect and classify objects in crowded and non-crowded scenes. It can also display live counters for persons, bicycles, and vehicles. In crowded videos, the output may become visually busy because many objects are detected in the same frame. To improve the visual quality, the confidence threshold and minimum box area can be increased, while optional IDs and center points can be hidden.

The project demonstrates that combining AI-based detection with traditional computer vision improves the final system. YOLOv8 provides object recognition, while OpenCV provides motion analysis, tracking, counting, and shape-based processing.

Challenges

Crowded Scenes

Many objects in the same frame can make the output crowded.

Double Counting

Tracking reduces this problem, but very crowded scenes can still be challenging.

Lighting Conditions

Poor lighting affects both detection and motion masks.

Hough Sensitivity

Circle and line detection depends on good video quality and parameter tuning.

Future Improvements

- Use DeepSORT for stronger tracking in crowded scenes.
- Add speed estimation for vehicles.
- Improve line-crossing logic using direction zones.
- Add a Streamlit web interface for easier demonstration.
- Use GPU acceleration for faster real-time processing.

Conclusion

The final system successfully combines the two project ideas into one professional computer vision application. It performs real-time object detection, motion detection, tracking, counting, and shape analysis. The implementation satisfies the requirements of both options and provides a strong practical demonstration of intelligent surveillance and traffic analysis.

Final Result: The project shows how YOLOv8 and OpenCV can work together to build a complete intelligent traffic surveillance and object analysis system.